



NAME : HAMDAN ALI

WEEK 7 : Day 34

DAY 34

TASK : Screenshoots Of Output

CODEBASE Of the Work

The screenshot displays a web browser window at `http://localhost:5173/login`. The page features a "Welcome back" heading and a prompt to "Log in to see your tasks." Below this are input fields for "Email" and "Password", followed by a blue "Log in" button. A link for "Don't have an account? [Create one](#)" is positioned at the bottom of the login form.

The background shows a VS Code editor with a file explorer on the left. The project structure includes folders for "Week1" through "Week7", with "day31", "day32", and "day33" expanded under "Week7". The terminal at the bottom shows the following commands and output:

```
PS D:\internship ( HindukushSoft Technologies )\Week7\taskflow-frontend> npm run dev
> taskflow-frontend@0.0.0 dev
> vite

2:13:27 PM [vite] (client) Re-optimizing dependencies because lockfile has changed

VITE v8.2.2 ready in 722 ms

→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```



Full PROJECT View

Create your account

Name

Email

Password

Confirm password

Sign up

Already have an account? [Log in](#)

Welcome back

Log in to see your tasks.

Email

demo@example.com

Password

.....

Log in

Don't have an account? [Create one](#)



localhost:5173/dashboard

TaskFlow

New Task

Title

Description

Status

Pending

Priority

Medium

Create Task

My Tasks

Search by title...

All statuses

All priorities

Save password?

Username

demo@example.com

Password

password123

Never

Save

No thanks

Passwords are saved to [Password Manager](#) on this device.

Hi, Demo User

Log out



EXPLANATION OF THE TASK

DAY 34

TASKFLOW

Day 34

Authentication, Protected Routes And User-Scoped Data

Week 7 — Day 34

Task: Protect Routes and Show User-Specific Data

Learning Topics: Authentication Flow (Frontend + Backend)

Stack: Laravel 12 + Sanctum, React 18, React Router

1. Introduction And Objective

Day 34's learning objective was authentication flow across both the frontend and backend: protecting API routes so only logged-in users can reach them, and ensuring each user only ever sees their own data. This is the task that turns TaskFlow from a shared, public demo into a genuine multi-user application.

I implemented token-based authentication using Laravel Sanctum on the backend — the officially recommended, lightweight approach for a decoupled SPA talking to a Laravel API — paired with a React Context + React Router setup on the frontend that protects routes and automatically attaches the user's token to every request.

2. Backend: Sanctum Token Authentication

2.1 Why Sanctum (and not sessions or a full OAuth server)

Laravel offers several authentication approaches. Sanctum was the right choice here because TaskFlow's React frontend is a genuinely separate client (it could be hosted on a completely different domain), and Sanctum's personal access tokens give a simple, stateless "Authorization: Bearer <token>" flow without the operational overhead of running a full OAuth2 server like Passport — this matches exactly how the SuperTokens and JWT/role-based-access articles I reviewed describe protecting routes for SPA + API architectures.

2.2 The Authentication Endpoints

Method	Endpoint	Access	Purpose
POST	/api/register	Public	Create an account, returns a token
POST	/api/login	Public	Verify credentials, returns a token
GET	/api/me	Protected	Return the authenticated user (session re-hydration)
POST	/api/logout	Protected	Revoke the token used for this request

2.3 Registration And Login Logic

`AuthController::register()` validates the incoming data (unique email, minimum password length, password confirmation), hashes the password with `Hash::make()` — Laravel never stores plain-text passwords — and issues a token via `$user->createToken('taskflow-web')->plainTextToken`. `login()` deliberately returns the same generic "credentials are incorrect" error whether the email doesn't exist or the password is wrong, which prevents an attacker from using the login form to enumerate registered email addresses, a real security consideration I wanted to demonstrate awareness of rather than just get the happy path working.

2.4 Protecting Routes with Middleware

In `routes/api.php`, every task-related route is nested inside `Route::middleware('auth:sanctum')->group(...)`. This single middleware guard rejects any request without a valid, unexpired token with a clean 401 JSON response before it ever reaches `TaskController` — the controller code itself never has to think about "is this user logged in?".

```
Route::middleware('auth:sanctum')->group(function () {  
    Route::get('/me', [AuthController::class, 'me']);  
    Route::post('/logout', [AuthController::class, 'logout']);  
    Route::apiResource('tasks', TaskController::class);  
});
```

2.5 Showing User-Specific Data — Defense in Depth

Being logged in is not the same as being allowed to see a specific piece of data. I scoped every single Task query to the authenticated user with a dedicated `ownedBy()` query scope, and additionally added an explicit `authorizeOwnership()` check inside `show()`, `update()` and `destroy()` that throws an `AuthorizationException` (mapped to a 403 response) if a user tries to access a task that belongs to someone else. This "belt and braces" approach means that even if the query scope were ever forgotten on a future endpoint, a user still cannot read or modify another user's tasks — an important, realistic security habit rather than relying on a single point of protection.

```
private function authorizeOwnership(Request $request, Task $task): void  
{  
    if ($task->user_id !== $request->user()->id) {  
        throw new AuthorizationException('You do not have permission to access this task.');    }  
}
```

2.6 Schema Change: Enforcing Ownership at the Database Level

A follow-up migration tightens the `tasks.user_id` column from nullable to NOT NULL and adds a real foreign key constraint back to users, with `cascadeOnDelete()` so a user's tasks are cleanly removed if their account is ever deleted. This mirrors how a real team would evolve a schema incrementally as new requirements (authentication) arrive, rather than rebuilding everything from scratch.

3. Frontend: Auth Context, Protected Routes And Token Handling

3.1 AuthContext — A Single Source of Truth

`AuthProvider` (`src/context/AuthContext.jsx`) wraps the whole app and exposes `{ user, isAuthenticated, isLoading, login, register, logout }` via the `useAuth()` hook. On first mount it checks `localStorage` for a previously saved token and — rather than blindly trusting it — verifies it against `GET /api/me`. If the token has been revoked or expired server-side, the stale local state is cleared quietly instead of leaving the UI in a broken, "logged in but every request fails" limbo.

3.2 Automatic Token Attachment via an Axios Interceptor

Instead of manually adding an `Authorization` header to every single API call, a single `Axios` request interceptor reads the token from `localStorage` and attaches it to every outgoing request. A matching response interceptor watches for 401 responses (token missing, expired, or revoked) and automatically clears local auth state and redirects to the login page — so an expired session degrades gracefully instead of the app silently failing on every request.

```
api.interceptors.request.use((config) => {  
  const token = localStorage.getItem('taskflow_token');  
  if (token) config.headers.Authorization = `Bearer ${token}`;  
  return config;  
});
```

3.3 Protecting Routes with React Router

`ProtectedRoute` is a small guard component used as a wrapper element inside the route tree. While auth status is still being resolved it shows a neutral "Checking your session..." state (avoiding a jarring flash of the login page for users who are actually already logged in); once resolved, it either

renders the protected page via `<Outlet />` or redirects to `/login`, remembering where the user was trying to go via location state so they land back there after logging in.

```
<Route element={<ProtectedRoute />}>
  <Route path="/dashboard" element={<Dashboard />} />
</Route>
```

3.4 Login And Register Forms

Both forms are simple controlled components that call `authService.login()` / `authService.register()`, surface any validation errors returned by Laravel directly beneath the relevant field, and redirect to the dashboard on success using the location state captured by `ProtectedRoute`.

4. End-to-End Flow

1. User submits the registration form -> POST `/api/register` -> Laravel creates the user and returns a token.
2. The token is stored in `localStorage` and the user object is placed into `AuthContext`.
3. Every subsequent request (e.g. GET `/api/tasks`) automatically carries `Authorization: Bearer <token>` via the Axios interceptor.
4. Laravel's `auth:sanctum` middleware validates the token; `TaskController` further scopes every query to `request()->user()->id`.
5. On logout, the specific token is revoked server-side (`currentAccessToken()->delete()`) and cleared from `localStorage`.

5. Challenges And How I Solved Them

- Avoiding a flash of the login page for already-authenticated users on refresh — solved with an explicit `isLoading` bootstrap phase in `AuthContext` that blocks rendering of protected content until the stored token has actually been verified against the server.
- Preventing a user from spoofing another user's ID when creating a task — solved by never trusting a client-supplied `user_id`; it is always derived server-side from `$request->user()->id`.
- Keeping the frontend resilient to an expired/revoked token — solved with a global response interceptor that detects 401s anywhere in the app and cleanly logs the user out instead of leaving broken, half-authenticated UI state.



6. Key Takeaways

- Token-based authentication (Sanctum) is a clean, stateless fit for a decoupled React + Laravel architecture.
- Route protection should happen in at least two layers: middleware for "are you logged in", and explicit ownership checks for "are you allowed to see this specific record".
- Never trust client-supplied identifiers for ownership — always derive them from the authenticated session server-side.
- A dedicated Axios interceptor is far more maintainable than manually attaching auth headers throughout an app.

WHERE TO FIND THE CODE

All source code for this task is included alongside this document inside the backend/ and frontend/ folders.

THANK YOU!